

src/model.py

209 lines

```
1 """
2 model.py
3 =====
4
5 The CONTROL-ORIENTED dynamic model of the well, identified from open-loop
6 step tests (see `scripts/03_identify_model.py`).
7
8 Model form
9 -----
10 For each output y in {Q, WHP, FLP, BHP}:
11
12  $y(k+1) = y(k) + a_y * (y_{ss}(u(k)) + d_y - y(k))$ ,  $a_y = 1 - e^{(-Ts/\tau_y)}$ 
13
14 i.e. a **first-order lag towards an identified steady-state characteristic**,
15 plus a constant output disturbance  $d_y$  estimated online.
16
17 Why this form
18 -----
19 * The step tests show the process is strongly NONLINEAR: the steady-state gain
20  $dQ/du$  is ~3.6 bbl/hr per % near 20 % opening but only ~0.15 at 90 %. A
21 single fixed-gain linear model would badly mispredict at one end of the
22 range or the other.
23 * Splitting the model into "where does it end up" (the steady-state curve
24  $y_{ss}(u)$ , read directly off the step-test end points) and "how fast does it
25 get there" (one time constant per output) keeps it completely explainable -
26 every number in the model is something an engineer can point at on a plot -
27 while capturing the nonlinearity exactly where it matters.
28 * It reduces exactly to a classical first-order-plus-gain model on any small
29 interval: locally,  $y_{ss}(u)$  is a straight line of slope  $K(u)$ , so the model is
30  $y(k+1) = y(k) + a*(K*du + y_0 - y(k))$ . The per-region gains  $K(u)$  are
31 reported in the engineering report.
32 * Dead time was fitted and came out at ~0 steps (the plant has no transport
33 delay), so  $\theta$  is carried in the model but is zero here.
34
35 Assumptions and limitations
36 -----
37 * Steady-state curve is piecewise linear between the identified step end
38 points; between knots the true curve is slightly convex, which is the main
39 source of prediction error (quantified in the validation report).
40 * One time constant per output, assumed independent of operating point and of
41 step direction. The step tests support this (spread of fitted  $\tau$  across
42 steps is small - see the report).
43 * No dead time, no interaction beyond what is implicit in the shared
44 dependence on  $u$ , and no reservoir depletion over the horizon.
45 * Valid over 0-100 % choke, the range the step tests covered. Extrapolation
46 beyond the knots is clamped (flat), which is flagged by `is_extrapolating`.
47 * The model is only ever used over a 5-step prediction horizon, where these
48 approximations are small; the online disturbance estimate  $d_y$  removes any
49 residual steady-state offset.
50 """
51
52 from __future__ import annotations
53
54 import json
55 from dataclasses import dataclass, field
56
57 from pathlib import Path
58 from typing import Dict, List, Sequence
59
60 import numpy as np
61
62 OUTPUTS = ("Q", "WHP", "FLP", "BHP")
63
64 @dataclass
65 class OutputModel:
66     """Identified model of one measured output."""
67
```

```

68 name: str
69 tau: float # first-order time constant [h]
70 theta: float # dead time [h] (0 for this plant)
71 u_knots: List[float] # choke openings at which steady state is known
72 y_knots: List[float] # corresponding steady-state values
73 tau_per_step: List[float] = field(default_factory=list) # diagnostics
74 rmse_fit: float = 0.0
75
76 # -- steady-state characteristic -----
77 def y_ss(self, u: float | np.ndarray) -> float | np.ndarray:
78     """Identified steady-state value of this output at choke opening u."""
79     return np.interp(u, self.u_knots, self.y_knots)
80
81 def gain(self, u: float, du: float = 1.0) -> float:
82     """Local steady-state gain dy/du [output units per % choke]."""
83     return float((self.y_ss(u + du / 2) - self.y_ss(u - du / 2)) / du)
84
85 def alpha(self, ts: float) -> float:
86     """Discrete lag coefficient for a control interval of `ts` hours."""
87     return float(1.0 - np.exp(-ts / self.tau))
88
89 def is_extrapolating(self, u: float) -> bool:
90     return u < min(self.u_knots) - 1e-9 or u > max(self.u_knots) + 1e-9
91
92 def to_dict(self) -> dict:
93     return {
94         "name": self.name,
95         "tau": self.tau,
96         "theta": self.theta,
97         "u_knots": list(map(float, self.u_knots)),
98         "y_knots": list(map(float, self.y_knots)),
99         "tau_per_step": list(map(float, self.tau_per_step)),
100         "rmse_fit": float(self.rmse_fit),
101     }
102
103 @staticmethod
104 def from_dict(d: dict) -> "OutputModel":
105     return OutputModel(
106         name=d["name"],
107         tau=d["tau"],
108         theta=d["theta"],
109         u_knots=d["u_knots"],
110         y_knots=d["y_knots"],
111         tau_per_step=d.get("tau_per_step", []),
112         rmse_fit=d.get("rmse_fit", 0.0),
113     )
114
115
116 @dataclass
117 class WellModel:
118     """The full four-output identified model used by the MPC."""
119
120     ts: float
121     outputs: Dict[str, OutputModel]
122
123     # -----
124     # Prediction
125     # -----
126     def predict(
127         self,
128         y0: Dict[str, float],
129         u_sequence: Sequence[float],
130         disturbance: Dict[str, float] | None = None,
131     ) -> Dict[str, np.ndarray]:
132         """
133         Roll the model forward.
134
135         Parameters
136         -----
137         y0 : dict

```

```

138 Current measured values {Q, WHP, FLP, BHP} - the prediction always
139 starts from the real plant measurement, so errors cannot accumulate
140 across control intervals.
141 u_sequence : sequence of float
142 Choke opening applied at each step of the horizon.
143 disturbance : dict, optional
144 Constant output disturbance d_y added to the steady-state target,
145 estimated online from the one-step prediction error. This is what
146 removes steady-state offset caused by plant/model mismatch.
147
148 Returns
149 -----
150 dict of arrays, each of length len(u_sequence), giving the predicted
151 trajectory of each output (the value at the END of each step).
152 """
153 d = disturbance or {k: 0.0 for k in OUTPUTS}
154 y = {k: float(y0[k]) for k in OUTPUTS}
155 traj = {k: np.empty(len(u_sequence)) for k in OUTPUTS}
156
157 for i, u in enumerate(u_sequence):
158     for k in OUTPUTS:
159         m = self.outputs[k]
160         target = float(m.y_ss(u)) + d.get(k, 0.0)
161         y[k] += m.alpha(self.ts) * (target - y[k])
162         traj[k][i] = y[k]
163 # A naturally flowing well cannot produce a negative rate.
164 traj["Q"] = np.maximum(traj["Q"], 0.0)
165 return traj
166
167 def predict_one(
168     self,
169     y0: Dict[str, float],
170     u: float,
171     disturbance: Dict[str, float] | None = None,
172 ) -> Dict[str, float]:
173     """One-step-ahead prediction (used by the disturbance estimator)."""
174     traj = self.predict(y0, [u], disturbance)
175     return {k: float(v[0]) for k, v in traj.items()}
176
177 # -----
178 # Persistence
179 # -----
180 def save(self, path: str | Path) -> None:
181     payload = {
182         "ts": self.ts,
183         "outputs": {k: m.to_dict() for k, m in self.outputs.items()},
184     }
185     Path(path).write_text(json.dumps(payload, indent=2))
186
187 @staticmethod
188 def load(path: str | Path) -> "WellModel":
189     payload = json.loads(Path(path).read_text())
190     return WellModel(
191         ts=payload["ts"],
192         outputs={
193             k: OutputModel.from_dict(v) for k, v in payload["outputs"].items()
194         },
195     )
196
197 # -----
198 # Reporting helper
199 # -----
200 def gain_table(self, u_points: Sequence[float]) -> "list[dict]":
201     """Local steady-state gains at several operating points, for the report."""
202     rows = []
203     for u in u_points:
204         row = {"choke_pct": u}
205         for k in OUTPUTS:
206             row[f"K_{k}"] = self.outputs[k].gain(u, du=2.0)
207     rows.append(row)

```

```
208 return rows
209
```